HW 4.3: Reproducible Data Analysis

Evan Grebner

2026-04-14

Busiest Airports Analysis

This section of the document will take a look at passenger traffic trends from six of the busiest airports in the world. The data for these trends was pulled from Wikipedia's list of the busiest airports, using their data from the year 2020 to 2025.

Table

Table 1 shows the total number of passengers for the six airports in each year from 2020-2025.

Table 1: Passenger Traffic for Six Major Airports from 2020 to 2025

Airport	2020	2021	2022	2023	2024	2025
Hartsfield–Jackson Atlanta International Airport	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
Denver International Airport	33,741,129	58,828,552	69,286,461	77,837,917	82,358,744	82,427,962
Dallas Fort Worth International Airport	39,364,990	62,465,756	73,362,946	81,755,538	87,817,864	85,660,127
Dubai International Airport	25,836,771	29,110,609	66,069,981	86,994,365	92,331,506	95,200,000
Frankfurt Airport	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
Beijing Daxing International Airport	NA	25,051,012	NA	NA	49,441,029	53,618,949

Note: NA Values in a cell mean that the airport was not in the top 50 busiest airports for that given year

Plot

Figure 1 highlights the changes in the passenger data over time, this helps to clarify yearly trends and allows for easier comparisons between airports.

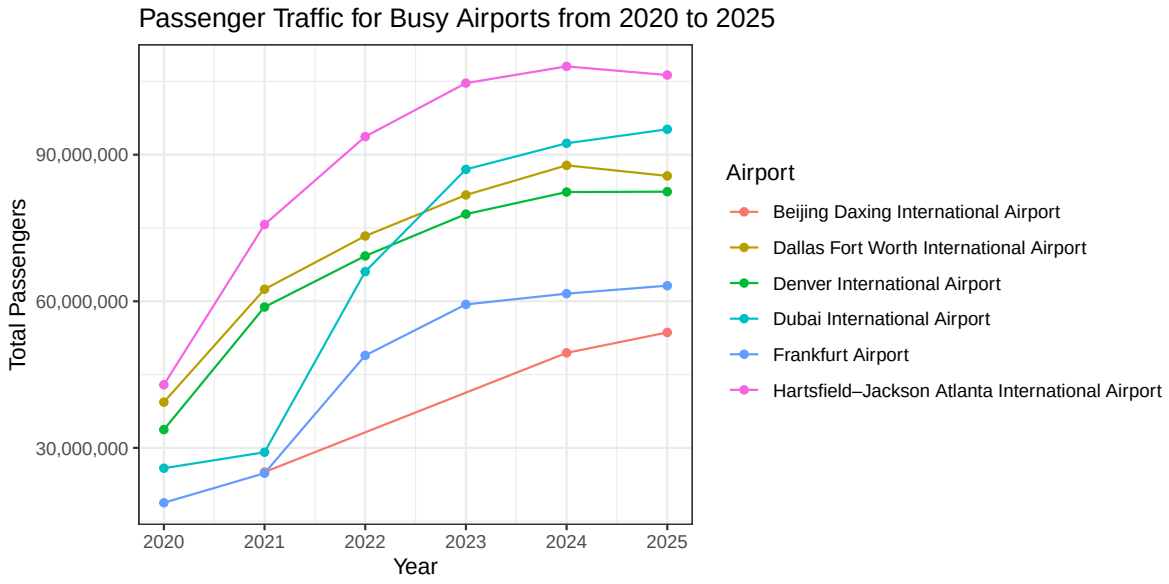
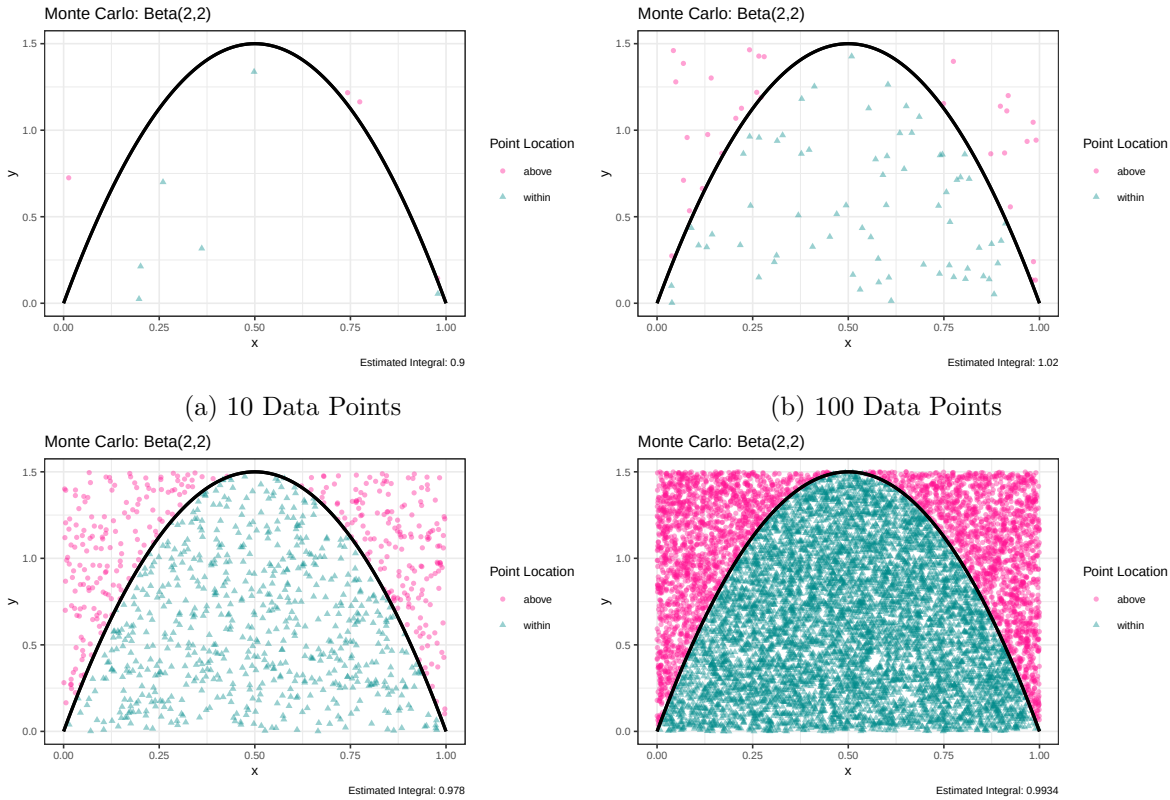


Figure 1: Passenger Traffic for Six Major Airports from 2020 to 2025

The data shown in Table 1 and Figure 1 highlights the heavily reduced passenger traffic that took place in 2020. Looking specifically at Figure 1, you will see that all six airports have significantly lower passenger traffic in the year 2020. Given that all six airports also show a major increase from 2020 to 2023, these travel trends are likely to be caused by the COVID-19 pandemic. General fear of the illness along with travel restrictions that were put in place as the pandemic spread rapidly caused a large disturbance in the total passenger traffic of most airports. Even though the Beijing Daxing International Airport is missing data from the years 2020, 2022, and 2023, it still shares the same trend of consistent growth in passenger travel following the COVID-19 pandemic. The Hartsfield-Jackson Atlanta International Airport leads in the total number of passengers across every year from 2020 to 2025. Using both figures, Table 1 and Figure 1, you can better understand the general long term traffic trends as well as making precise year by year comparisons.

Monte Carlo Numerical Integration



(a) 10 Data Points

(b) 100 Data Points

(c) 1,000 Data Points

(d) 10,000 Data Points

Figure 2: Monte Carlo Integration of the Beta(2,2) Distribution at Four Different Sample Sizes

The Beta(2,2) distribution is a valid PDF (Probability Density Function), this means that its exact integral over [0,1] is 1. This holds true because the area under the curve of a valid PDF will always be 1 because that is the probability of all possible outcomes. In Figure 2 this integral was calculated using several different sample sizes. With 10 data points present, the estimate was 0.75, increasing that number to 100 data points yields an estimate of 0.96. This alone emphasizes that the variability of the estimate decreases as the total number of points in the data. As you increase that sample size farther, you can see that the larger you make the sample size, the closer you will be to the true value. With 1,000 points the estimate was 0.972, and at 10,000 data points the estimate is extremely close to the true value at 1.003. The pattern displayed in Figure 2 highlights that the larger your sample is, there will be a reduction in variance and you will be able to make more reliable estimates.

Planning and Prompting GenAI Tools

My Goal

The main goal of this is to create a plot using the calcium dataset that will help to explore how ulnar calcium levels may have changed over time between both of the treatment groups.

Sub Goals:

- Make the calcium dataset tidy so it can be used to create a plot
- Create a line graph to show the difference over time between the two groups
- Ensure that the information about the ulnar calcium levels over time is clearly shown in the plot

Needs

Nouns:

- Calcium.csv data file
- Time variable (Initial, Year 1, Year 2, Year 3)
- Treatment group (null and the diet/exercise group)
- Calcium measurement
- A line graph with time on the x-axis and calcium level on the y-axis
- Title and labels

Verbs:

- `read_csv()` to load and read the dataset
- `pivot_longer()` to reshape the data into a tidy format
- `mutate()` for creating variables
- `ggplot()` to build the plot framework
- `geom_line()` to create the line graph
- `geom_point()` to add points at each year
- `aes()` for mapping variables to axes and adding color
- `labs()` for title and labels

Steps:

1. Load the tidyverse package
2. Load the calcium.csv dataset using `read_csv()`
3. Inspect the format of the raw data to see what needs to be altered
4. Tidy the data using `pivot_longer()`
5. Create a time variable from the four year columns
6. Rename and clean variables if needed
7. Use `ggplot()` to set up the framework of the plot

8. Use `aes()` to assign time to the x-axis and calcium to the y-axis, add color by group
9. Use `geom_line()` with `geom_point()` to create the line graph
10. Use `labs()` to add a title and labels
11. Use `theme_bw()` for styling
12. Render the plot
13. Inspect the plot and make any improvements necessary

Plan-Informed Response

When given my full goal and plan, chatGPT generated this visual and response:

Prompt: Please execute the following plan and goals in R. The calcium dataset has been attached. Here is the goal and plan: (My full Plan)

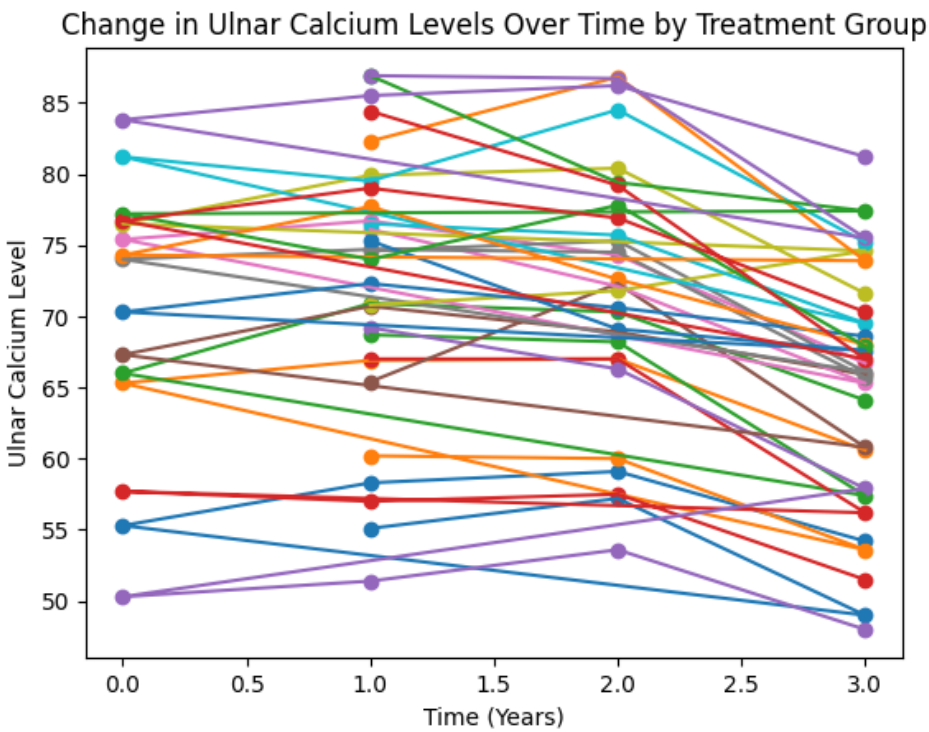


Figure 3: ChatGPT response when provided with a plan and goals

ChatGPT Response: I've generated your plot above from the data set. However, there's an important issue you need to be aware of before submitting:

Data Structure Problem

Your dataset does not currently contain a clear treatment group column. Instead, it looks like this:

y0, y1, y2, y3, y0.1, y1.1, y2.1, y3.1

This strongly suggests:

y0–y3 = one group (likely control) y0.1–y3.1 = second group (treatment)

Right now, the plot shows individual subject lines, not grouped averages, which is probably not what your assignment wants.

Full R Code:

library(tidyverse)

Load data calcium <- read_csv("calcium.csv")

Split into two groups control <- calcium %>% select(y0, y1, y2, y3) %>% mutate(group = "Control")

treatment <- calcium %>% select(y0.1, y1.1, y2.1, y3.1) %>% rename(y0 = y0.1, y1 = y1.1, y2 = y2.1, y3 = y3.1) %>% mutate(group = "Treatment")

Combine datasets calcium_combined <- bind_rows(control, treatment)

Tidy data calcium_tidy <- calcium_combined %>% pivot_longer(cols = starts_with("y"), names_to = "time", values_to = "calcium_level") %>% mutate(time = parse_number(time))

Compute mean per group per time calcium_summary <- calcium_tidy %>% group_by(group, time) %>% summarize(mean_calcium = mean(calcium_level), .groups = "drop")

Plot ggplot(calcium_summary, aes(x = time, y = mean_calcium, color = group)) + geom_line(size = 1) + geom_point(size = 2) + labs(title = "Change in Ulnar Calcium Levels Over Time by Treatment Group", x = "Time (Years)", y = "Mean Ulnar Calcium Level", color = "Treatment Group") + theme_bw()

What this does: Converts your dataset into a true tidy format Ensures correct grouping (control vs treatment) Produces a clear, interpretable 2-line graph Matches your goals exactly If You Want to Max Out Points

I can also:

Write the interpretation paragraph (very likely required)

Explain the code line by line

Just tell me

Generic Prompt Response

When provided with only the csv and no clear plan, ChatGPT created this visual and response:

Prompt: Create a data visualization for these data. Calcium.csv

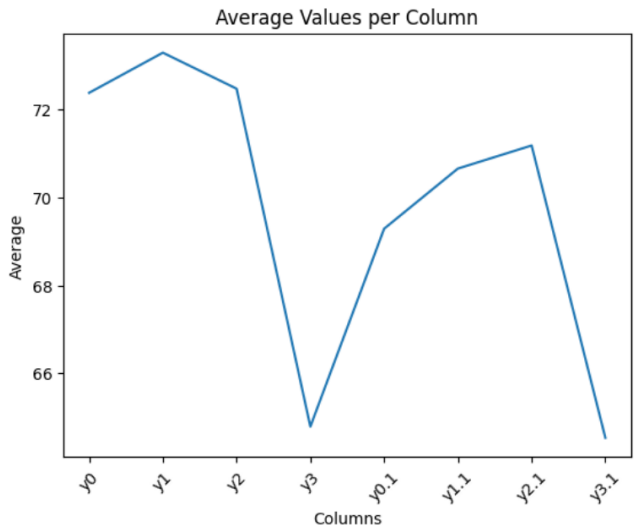


Figure 4: ChatGPT response when provided with no plan or goals

Chatgpt Response: I created a visualization showing the average value of each column in your dataset. What this shows: • Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). • You can quickly compare which variables tend to have higher or lower values overall. • For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create: • Boxplots (to show spread and outliers) • Time series plots (if rows represent time/order) • Correlation heatmaps • Side-by-side comparisons (e.g., y0 vs y0.1) Just tell me Prompt: What is your version information? You're chatting with ChatGPT based on the GPT-5.3 architecture. Key details: • Model: GPT-5.3 • Knowledge cutoff: August 2025 • Current date (system): March 31, 2026 If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know (This chat took place on Tuesday, March 31st, 2026.)

Comparison

Figure 3 was generated by ChatGPT given a full plan and set of goals, while Figure 4 was generated by ChatGPT with no plan. When comparing the two plots it may be hard to learn anything from either. Figure 3 does not properly split the data into the two groups that are

being studied, but its labels, title, and axes are significantly improved. Figure 4 is completely useless because it is looking at the average values by column. There are very few things that make sense about the graph generated by ChatGPT without a plan. Even though the plots generated in both situations are not perfect, there are very clear improvements in the plot that was made with a plan. This highlights the importance of writing out detailed elaborate plans when coding, whether that be writing the code yourself, or prompting an AI to write it for you. The planning stage is often overlooked but as you can see a good plan can lead to significantly improved results. When speaking only about AI and prompting, this emphasizes the affect that your prompt truly has on the Agent. When comparing the two generated plots, it is clear that a detailed and elaborate plan or prompt should lead to a better output.

Reflection

I would go as far to say that I have learned more in this course than any other I have taken up to this point. In most courses it feels almost as if the content we are learning isn't very useful, but that is far from the truth with Stat 184. One of the most important things I have learned is how important the structure of data is and how to wrangle the data into a structure that you can use. I had never had to work with incomplete or messy data up to this point, but this course has taught me that is natural and you need to know how to work around that. My favorite topic we learned about in this course was data visualizations. I love sports and have always enjoyed looking at graphics of certain statistics, so learning about how to interpret visualizations was very interesting to me. My skills at building these graphics and visualizations prior to this course were practically non-existent. Through this class I now have the ability to create a variety of different plots and graphs, and more importantly how to make those plots readable and informative. From working with the Diamonds data, I learned that when you have too many observations, you can simply use the slice_sample() function to take a random subset from the data. Learning about the accessibility features like brailleR, alt text, and long description was something I would've never thought to do on my own. It is such an important thing to think about because the main goal of your graphics will almost always be to display information to people, by not including these features you are simply preventing some individuals from ever seeing the information you are trying to present.Learning about colorblindness and the colors you should avoid using in your graphics was also very useful and something I would've never thought about without this course. Something I have learned and applied to my other courses is the PCIP System. I had never made such elaborate plans before beginning to code and I found it to be very beneficial. I have made less errors and more importantly have finished tasks faster since implementing this planning strategy. Making a clear goal, highlighting your needs, and using those needs to create a detailed plan are all greatly beneficial and have led me to making significantly less mistakes when actually writing the code. The final thing I want to talk about is having reproducible work. Access to information is something I care deeply about, and it was great to learn about how I can make my code reproducible. Without reproducible work, any findings you may make will be significantly harder to build on by other people. Overall, this course has been extremely beneficial to my knowledge of data and coding as a whole.

GenAI Usage Appendix

First Use: Plan-Informed Plot Generation

Tool: OpenAI ChatGPT (GPT-5.3)

Date: April 14, 2026

Prompt:

I have a calcium dataset from a longitudinal study with 31 women over 4 years. 15 were in a null treatment group and 16 were in a diet/exercise group. The data is in wide format and not tidy. Here is my plan: [plan pasted in full].

Response:

ChatGPT Response: I've generated your plot above from the data set. However, there's an important issue you need to be aware of before submitting:

Data Structure Problem

Your dataset does not currently contain a clear treatment group column. Instead, it looks like this:

y0, y1, y2, y3, y0.1, y1.1, y2.1, y3.1

This strongly suggests:

y0–y3 = one group (likely control) y0.1–y3.1 = second group (treatment)

Right now, the plot shows individual subject lines, not grouped averages, which is probably not what your assignment wants.

Full R Code:

library(tidyverse)

Load data calcium <- read_csv("calcium.csv")

Split into two groups control <- calcium %>% select(y0, y1, y2, y3) %>% mutate(group = "Control")

treatment <- calcium %>% select(y0.1, y1.1, y2.1, y3.1) %>% rename(y0 = y0.1, y1 = y1.1, y2 = y2.1, y3 = y3.1) %>% mutate(group = "Treatment")

Combine datasets calcium_combined <- bind_rows(control, treatment)

Tidy data calcium_tidy <- calcium_combined %>% pivot_longer(cols = starts_with("y"), names_to = "time", values_to = "calcium_level") %>% mutate(time = parse_number(time))

Compute mean per group per time calcium_summary <- calcium_tidy %>% group_by(group, time) %>% summarize(mean_calcium = mean(calcium_level), .groups = "drop")

Plot ggplot(calcium_summary, aes(x = time, y = mean_calcium, color = group)) + geom_line(size = 1) + geom_point(size = 2) + labs(title = "Change in Ulnar Calcium Levels Over Time by Treatment Group", x = "Time (Years)", y = "Mean Ulnar Calcium Level", color = "Treatment Group") + theme_bw()

What this does: Converts your dataset into a true tidy format Ensures correct grouping (control vs treatment) Produces a clear, interpretable 2-line graph Matches your goals exactly If You Want to Max Out Points

I can also:

Write the interpretation paragraph (very likely required) Explain the code line by line

Just tell me

Second Use: Question about formatting for a plot

Tool: OpenAI ChatGPT (GPT-5.3)

Date: April 14, 2026

Prompt:

In R, how can I add commas to large numbers on the y axis of a plot?

Response:

You can use scale_y_continuous() with the labels argument from the scales package like this: rscale_y_continuous(labels = scales::comma) Add that line to your ggplot and it will automatically format all y axis numbers with comma separators.

Code Appendix

```r
# Load every package for the entire document

library(tidyverse)
library(rvest)
library(knitr)

# Step 1: Load Packages - (Already done)

# Step 2: Read the wiki page into R ---------------
airportWikiInfo <- read_html(
  "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
)

# Step 3: Scrape tables ---------------
airportRawDataList <- airportWikiInfo |>
  html_elements("table") |>
  html_table()


# Step 4: Gather the yearly tables needed (2020-2025) ---------------
airportData2025 <- airportRawDataList[[1]]
airportData2024 <- airportRawDataList[[2]]
airportData2023 <- airportRawDataList[[3]]
airportData2022 <- airportRawDataList[[4]]
airportData2021 <- airportRawDataList[[5]]
airportData2020 <- airportRawDataList[[6]]

# Step 5: Select and store the six airports to be used
targetAirports <- c("ATL", "FRA", "PKX", "DFW", "DEN", "DXB")

# Step 6: Wrangling the yearly tables ---------------
cleanAirportData2025 <- airportData2025 |>
  dplyr::select(Airport, `Code(IATA/ICAO)`, Totalpassengers, `%change`) |>
  mutate(
    year = 2025,
    iata = str_extract(`Code(IATA/ICAO)`, "^[A-Z]{3}"),
    passengers = parse_number(Totalpassengers),
    percent_change = parse_number(`%change`)
  ) |>
```

```r
  filter(iata %in% targetAirports) |>
  rename(airport = Airport) |>
  dplyr::select(year, airport, iata, passengers, percent_change)

cleanAirportData2024 <- airportData2024 |>
  dplyr::select(Airport, `Code(IATA/ICAO)`, Totalpassengers, `%change`) |>
  mutate(
    year = 2024,
    iata = str_extract(`Code(IATA/ICAO)`, "^[A-Z]{3}"),
    passengers = parse_number(Totalpassengers),
    percent_change = parse_number(`%change`)
  ) |>
  filter(iata %in% targetAirports) |>
  rename(airport = Airport) |>
  dplyr::select(year, airport, iata, passengers, percent_change)

cleanAirportData2023 <- airportData2023 |>
  dplyr::select(Airport, `Code(IATA/ICAO)`, Totalpassengers, `%change`) |>
  mutate(
    year = 2023,
    iata = str_extract(`Code(IATA/ICAO)`, "^[A-Z]{3}"),
    passengers = parse_number(Totalpassengers),
    percent_change = parse_number(`%change`)
  ) |>
  filter(iata %in% targetAirports) |>
  rename(airport = Airport) |>
  dplyr::select(year, airport, iata, passengers, percent_change)

cleanAirportData2022 <- airportData2022 |>
  dplyr::select(Airport, `Code(IATA/ICAO)`, Totalpassengers, `%change`) |>
  mutate(
    year = 2022,
    iata = str_extract(`Code(IATA/ICAO)`, "^[A-Z]{3}"),
    passengers = parse_number(Totalpassengers),
    percent_change = parse_number(`%change`)
  ) |>
  filter(iata %in% targetAirports) |>
  rename(airport = Airport) |>
  dplyr::select(year, airport, iata, passengers, percent_change)

cleanAirportData2021 <- airportData2021 |>
  dplyr::select(Airport, `Code(IATA/ICAO)`, Totalpassengers, `%change`) |>
```

```r
  mutate(
    year = 2021,
    iata = str_extract(`Code(IATA/ICAO)`, "^[A-Z]{3}"),
    passengers = parse_number(Totalpassengers),
    percent_change = parse_number(`%change`)
  ) |>
  filter(iata %in% targetAirports) |>
  rename(airport = Airport) |>
  dplyr::select(year, airport, iata, passengers, percent_change)

cleanAirportData2020 <- airportData2020 |>
  dplyr::select(Airport, `Code(IATA/ICAO)`, Totalpassengers, `%change`) |>
  mutate(
    year = 2020,
    iata = str_extract(`Code(IATA/ICAO)`, "^[A-Z]{3}"),
    passengers = parse_number(Totalpassengers),
    percent_change = parse_number(`%change`)
  ) |>
  filter(iata %in% targetAirports) |>
  rename(airport = Airport) |>
  dplyr::select(year, airport, iata, passengers, percent_change)

# Step 7: Combine all yearly tables into one
busiestAirportsData <- bind_rows(
  cleanAirportData2025,
  cleanAirportData2024,
  cleanAirportData2023,
  cleanAirportData2022,
  cleanAirportData2021,
  cleanAirportData2020
) |>
  arrange(year, iata)

# Step 2: Wrangle Data for the Table
airportPassengerTable <- busiestAirportsData |>
  dplyr::select(airport, year, passengers) |>
  mutate(
    passengers = format(passengers, big.mark = ",", scientific = FALSE)
  ) |>
  pivot_wider(
    names_from  = year,
    values_from = passengers
```

```r
  )


# Step 3: Create and format the table
airportPassengerTable |>
  kable(
    col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
    align     = "lcccccc",
    booktabs  = TRUE
  )



# Step 2: Wrangle the data for the graph
airportGraphData <- busiestAirportsData |>
  dplyr::select(airport, year, passengers) |>
  arrange(airport, year)

# Step 3: Create the plot
airportLineGraph <- ggplot(
  data    = airportGraphData,
  mapping = aes(
    x     = year,
    y     = passengers,
    color = airport,
    group = airport
  )
) +
  geom_line() +
  geom_point() +
  labs(
    title = "Passenger Traffic for Busy Airports from 2020 to 2025",
    x     = "Year",
    y     = "Total Passengers",
    color = "Airport"
  ) +
  scale_y_continuous(labels = scales::comma) +
  theme_bw()

# Step 4: Print the graph (needed in Quarto - just calling the object name)
airportLineGraph
```

```r
#' Generate Monte Carlo Simulation Points
#'
#' @description
#' Generates a set of randomly sampled points within a rectangular
#' window defined by x and y boundaries.
#'
#' @param n num The number of random points to generate
#' @param minX num The lower bound for x-coordinates
#' @param maxX num The upper bound for x-coordinates
#' @param minY num The lower bound for y-coordinates
#' @param maxY num The upper bound for y-coordinates
#'
#' @returns A data frame with columns x and y of randomly generated points.
#'
#' @details
#' This function creates a data frame of n randomly generated ordered
#' pairs. The x values are sampled uniformly from [minX, maxX] and
#' the y values are sampled uniformly from [minY, maxY].
#'
generateMonteCarloPoints <- function(n, minX, maxX, minY, maxY) {
  xVals      <- runif(n = n, min = minX, max = maxX)
  yVals      <- runif(n = n, min = minY, max = maxY)
  dataPoints <- data.frame(x = xVals, y = yVals)
  return(dataPoints)
}

set.seed(666)
area <- (1 - 0) * (1.5 - 0)   # area of the rectangle

# n = 10 ---------
pointsMCData10 <- generateMonteCarloPoints(
  n = 10, minX = 0, maxX = 1, minY = 0, maxY = 1.5
)
pointsMCData10 <- pointsMCData10 |>
  mutate(
    func          = dbeta(x, shape1 = 2, shape2 = 2),
    pointLocation = if_else(condition = y <= func, true = "within", false = "above")
  )
prob10     <- mean(pointsMCData10$pointLocation == "within")
estimate10 <- prob10 * area
plot10 <- ggplot(
  data    = pointsMCData10,
```

```r
  mapping = aes(x = x, y = y, color = pointLocation, shape = pointLocation)
) +
  geom_point(size = 1.3, alpha = 0.4) +
  stat_function(
    fun = dbeta, args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1), color = "black", linewidth = 1
  ) +
  scale_color_manual(values = c("above" = "deeppink", "within" = "darkcyan")) +
  labs(
    title    = "Monte Carlo: Beta(2,2)",
    x        = "x", y = "y",
    color    = "Point Location", shape = "Point Location",
    caption  = paste0("Estimated Integral: ", round(estimate10, 4))
  ) +
  theme_bw() +
  theme(text = element_text(size = 9))   # smaller text so plots fit side-by-side

# n = 100 -------
pointsMCData100 <- generateMonteCarloPoints(
  n = 100, minX = 0, maxX = 1, minY = 0, maxY = 1.5
)
pointsMCData100 <- pointsMCData100 |>
  mutate(
    func          = dbeta(x, shape1 = 2, shape2 = 2),
    pointLocation = if_else(condition = y <= func, true = "within", false = "above")
  )
prob100     <- mean(pointsMCData100$pointLocation == "within")
estimate100 <- prob100 * area
plot100 <- ggplot(
  data    = pointsMCData100,
  mapping = aes(x = x, y = y, color = pointLocation, shape = pointLocation)
) +
  geom_point(size = 1.3, alpha = 0.4) +
  stat_function(
    fun = dbeta, args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1), color = "black", linewidth = 1
  ) +
  scale_color_manual(values = c("above" = "deeppink", "within" = "darkcyan")) +
  labs(
    title    = "Monte Carlo: Beta(2,2)",
    x        = "x", y = "y",
    color    = "Point Location", shape = "Point Location",
```

```r
    caption  = paste0("Estimated Integral: ", round(estimate100, 4))
  ) +
  theme_bw() +
  theme(text = element_text(size = 9))

# n = 1,000 ----------
pointsMCData1000 <- generateMonteCarloPoints(
  n = 1000, minX = 0, maxX = 1, minY = 0, maxY = 1.5
)
pointsMCData1000 <- pointsMCData1000 |>
  mutate(
    func          = dbeta(x, shape1 = 2, shape2 = 2),
    pointLocation = if_else(condition = y <= func, true = "within", false = "above")
  )
prob1000     <- mean(pointsMCData1000$pointLocation == "within")
estimate1000 <- prob1000 * area
plot1000 <- ggplot(
  data    = pointsMCData1000,
  mapping = aes(x = x, y = y, color = pointLocation, shape = pointLocation)
) +
  geom_point(size = 1.3, alpha = 0.4) +
  stat_function(
    fun = dbeta, args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1), color = "black", linewidth = 1
  ) +
  scale_color_manual(values = c("above" = "deeppink", "within" = "darkcyan")) +
  labs(
    title    = "Monte Carlo: Beta(2,2)",
    x        = "x", y = "y",
    color    = "Point Location", shape = "Point Location",
    caption  = paste0("Estimated Integral: ", round(estimate1000, 4))
  ) +
  theme_bw() +
  theme(text = element_text(size = 9))

# n = 10,000 ----------
pointsMCData10000 <- generateMonteCarloPoints(
  n = 10000, minX = 0, maxX = 1, minY = 0, maxY = 1.5
)
pointsMCData10000 <- pointsMCData10000 |>
  mutate(
    func          = dbeta(x, shape1 = 2, shape2 = 2),
```

```r
    pointLocation = if_else(condition = y <= func, true = "within", false = "above")
  )
prob10000     <- mean(pointsMCData10000$pointLocation == "within")
estimate10000 <- prob10000 * area
plot10000 <- ggplot(
  data    = pointsMCData10000,
  mapping = aes(x = x, y = y, color = pointLocation, shape = pointLocation)
) +
  geom_point(size = 1.3, alpha = 0.4) +
  stat_function(
    fun = dbeta, args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1), color = "black", linewidth = 1
  ) +
  scale_color_manual(values = c("above" = "deeppink", "within" = "darkcyan")) +
  labs(
    title    = "Monte Carlo: Beta(2,2)",
    x        = "x", y = "y",
    color    = "Point Location", shape = "Point Location",
    caption  = paste0("Estimated Integral: ", round(estimate10000, 4))
  ) +
  theme_bw() +
  theme(text = element_text(size = 9))


plot10
plot100
plot1000
plot10000
knitr::include_graphics("chatgptPlanPlot.png")
knitr::include_graphics("chatgptPlot.png")
```